1. Especificación del problema

BORRADOR ASISTIDO POR IA · REVISIÓN ACADÉMICA PENDIENTE

Maestría en Tecnologías de la Información · Análisis y Diseño de Algoritmos · Cohorte X, 2026-III. Integrantes: nombres y aprobación pendientes.

Se recibe una secuencia A de n enteros de Python, n >= 1. Se devuelve una estructura inmutable con suma, inicio i, fin j y subarreglo A[i:j+1], tal que 0 <= i <= j < n y su suma es máxima entre todos los intervalos contiguos no vacíos.

Desempate lexicográfico por (i,j): menor inicio y luego menor fin. No se modifica la entrada. Una entrada vacía produce ValueError; tipos no enteros, incluidos booleanos, producen TypeError. Todos negativos devuelve el mayor elemento, con el primer índice si se repite. Ejemplo: [-2,1,-3,4,-1,2,1,-5,4] produce suma 6, índices [3,6], subarreglo [4,-1,2,1].

2. Demostración de correctitud

Material de trabajo asistido por IA. El equipo debe reconstruir, comprender y verificar este argumento independientemente antes de entregarlo.

Definamos E(i) como la mejor suma de un intervalo no vacío que termina exactamente en i, y G(i) como la mejor suma de cualquier intervalo contenido en A[0:i+1]. La relación a revisar es E(0)=G(0)=A[0], E(i)=max(A[i], E(i-1)+A[i]) y G(i)=max(G(i-1), E(i)).

Invariante formal. Tras procesar i (0 <= i < n): current = E(i) = max_{0 <= l <= i} Σ_{k=l}^{i} A[k], y left es el menor l que alcanza E(i). Además best = G(i) = max_{0 <= l <= r <= i} Σ_{k=l}^{r} A[k], y (start,end) es el par lexicográficamente mínimo que alcanza G(i). Antes de la iteración siguiente estos predicados se mantienen sobre el prefijo ya procesado.

Inicialización. Antes del segundo elemento, current y best valen A[0]. El único intervalo es [0,0], de modo que ambos predicados se cumplen incluso para A[0] negativo.

Mantenimiento. Todo intervalo que termina en i es el singleton [i,i] o extiende algún intervalo terminado en i-1. Entre las extensiones, basta extender el mejor: sumar A[i] conserva su orden. Si current anterior es negativo conviene reiniciar; si es cero se extiende para conservar el inicio menor. Así current representa E(i), con el inicio más temprano. Todo óptimo global anterior permanece en el prefijo o termina en i. Comparar best con current establece G(i), resolviendo empates por (inicio,fin). La comparación explícita preserva también el fin más temprano para el mismo inicio.

Terminación. El índice avanza de 1 a n-1, con variante n-i que disminuye y está acotada inferiormente. Al salir, best representa G(n-1); los índices y la tupla devuelta satisfacen la postcondición. Para n=1 el ciclo no se ejecuta y la inicialización basta.

Fuerza bruta. Para un inicio fijo l, después de agregar A[r], current es la suma A[l:r+1]. Los dos bucles visitan cada par 0 <= l <= r < n. best conserva el mayor intervalo visitado; como los pares se recorren en orden lexicográfico y solo se actualiza ante mejora estricta, se respeta el mismo desempate. Ambos rangos son finitos.

3. Análisis de costo

Modelo RAM con enteros de tamaño acotado y operaciones aritméticas de costo constante. Los enteros Python de precisión arbitraria añaden costo dependiente del número de bits, por lo que las cotas en n requieren este supuesto.

Kadane: la operación dominante del núcleo es la actualización del mejor sufijo, exactamente K(n)=sumatoria de 1 para i=1,...,n-1 = n-1. Para n>=2, n/2 <= n-1 <= n: constantes explícitas c_inferior=1/2, c_superior=1 y n0=2 para este conteo. La validación hace n comprobaciones y la copia de salida hasta n elementos. El tiempo total permanece Θ(n), pero las constantes anteriores se refieren al conteo dominante, no a nanosegundos ni a todas las instrucciones Python.

Fuerza bruta: la operación dominante es current += A[r], exactamente B(n)=sumatoria de (n-l), l=0,...,n-1 = n(n+1)/2. Para n>=1, $n^2/2$ <= B(n) <= n^2: c_inferior=1/2, c_superior=1, n0=1. La inicialización, validación y copia final no cambian Θ(n^2).

Mejor y peor caso: Θ(n) para Kadane y Θ(n^2) para fuerza bruta; no hay salida anticipada y las cantidades de iteraciones no dependen de los valores. Para el caso promedio, bajo cualquier distribución de valores acotados condicionada a una longitud fija n y el mismo modelo RAM, esos conteos exactos son constantes respecto a la entrada; su esperanza es K(n) o B(n). Esto no predice un tiempo promedio exacto, pues cambian ramas, tamaño de enteros, caché y entorno.

Memoria auxiliar O(1) para ambos motores, excluyendo la salida de k elementos (O(k)). La traza educativa sí usa O(n) adicional y no interviene en el cronometraje. El experimento almacena entradas y resultados fuera del intervalo medido.

4. Implementación y pruebas

Ejecutar python -m pytest tests -q o python run_pipeline.py --qa-only. La evidencia medida está en results/qa_results.json con fecha UTC, commit si existe, hash del código fuente y salida capturada; no se afirma cobertura.

La suite contempla positivos, negativos, mixtos, ceros, singleton positivo y negativo, vacíos, repetidos, óptimos al inicio/final/centro y empates. Verifica suma, índices, salida y ausencia de mutación. Se comparan 500 entradas aleatorias de semilla 20261105 y los 1092 arreglos de longitudes 1 a 6 sobre {-1,0,1}, estos últimos también contra un oráculo independiente basado en sumas directas. La traza se contrasta con prefijos y sufijos calculados directamente.

El número de casos parametrizados Pytest difiere del número de arreglos explorados dentro de las pruebas diferenciales. La interfaz ejecuta exclusivamente la suite fija con un timeout de 45 segundos y no acepta código ni comandos externos.

Ejecución archivada: 44 casos Pytest, 44 aprobados, 0 fallidos, 0 errores; duración 2.949 s. Fecha UTC: 2026-11-05T21:48:52.603731+00:00.

Commit: 73739a6cfe4d2a80da37862755af77de1ee2d5df. SHA-256 del código: 8fdf0f9118ca668b903b7ee19598d25037db93bc832f53c50f81a064c158365e.

5. Experimento reproducible

Configuración oficial inicial: n=100,200,400,800,1600,3200; cinco repeticiones por tamaño y algoritmo; semilla 20261105; enteros uniformes en [-100,100]. Se generan todos los arreglos antes de medir y cada par de algoritmos recibe exactamente el mismo arreglo. Se alterna el orden por par para reducir sesgo, sin eliminarlo por completo. Se realiza una llamada de calentamiento por algoritmo.

Se mide una llamada real con perf_counter_ns; incluye contrato de validación y materialización de la salida, excluye generación, aserción de equivalencia, progreso, escrituras y gráficas. CSV: 60 observaciones; medianas calculadas por Pandas. Los metadatos JSON documentan Python, SO, procesador disponible, dependencias, fecha UTC, semilla, configuración, resolución del reloj, commit y hash fuente.

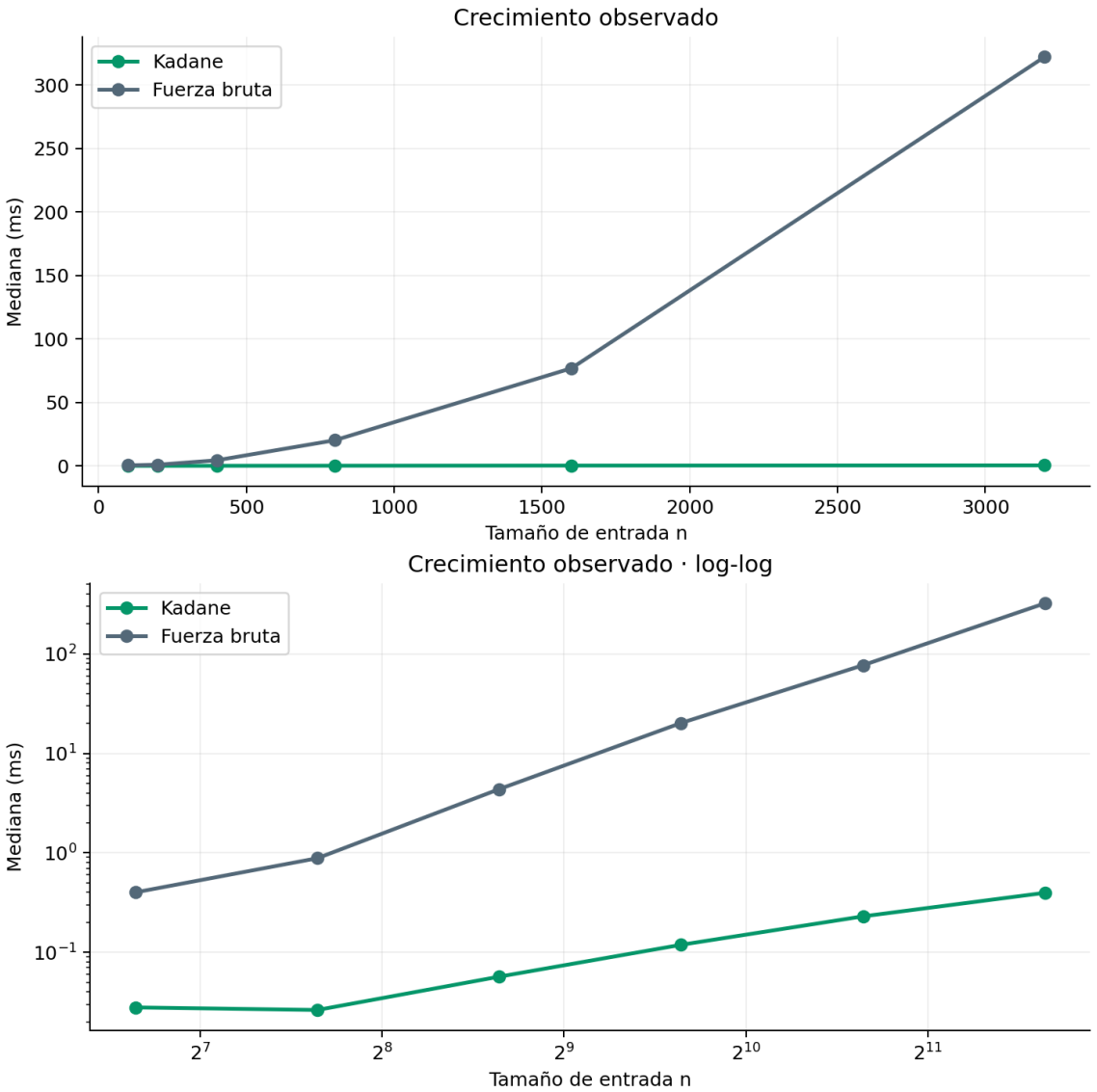
n	Kadane (ms)	Bruta (ms)	Razón K	Razón B
100	0.02790	0.401	-	-
200	0.02630	0.882	0.94	2.20
400	0.05690	4.370	2.16	4.95
800	0.11890	20.192	2.09	4.62
1600	0.22970	76.874	1.93	3.81
3200	0.39600	322.057	1.72	4.19

Entorno: Python 3.13.3; Windows-11-10.0.26200-SP0; procesador: Intel64 Family 6 Model 140 Stepping 1, GenuineIntel. Fecha UTC: 2026-11-05T21:48:54.900471+00:00.

Dependencias: streamlit 1.67.0; pandas 3.1.1; matplotlib 3.10.3; pytest 9.2.0

6. Contraste entre teoría y evidencia

Hipótesis teórica: al duplicar n, el factor de crecimiento temporal se aproxima a 2 para una solución lineal y a 4 para una cuadrática, cuando domina el término asintótico. Los cocientes observados se calculan sobre medianas reales, sin imponer esos valores. Las figuras normal y log-log se generan del CSV, no de curvas inventadas.



Figuras derivadas del CSV archivado. Interpretación y conclusión final pendientes del equipo; no constituyen demostración matemática.

7. Uso y verificación de IA

Herramienta: Codex, modelo identificado por el entorno como GPT-6; no se dispone de un identificador de compilación más preciso. Fecha de trabajo: 2026-11-05. Nivel propuesto: Ámbar (asistencia con código y pruebas); el equipo debe completar y aprobar su Declaración de Transparencia Tecnológica conforme a las indicaciones del curso.

Prompt: se conserva íntegro en docs/user_request.txt. Salidas relevantes: implementación inicial de referencia, suite QA, pipeline experimental, interfaz, documentación y propuesta de argumento de correctitud. No se atribuyen contribuciones a estudiantes ni se presenta la demostración como aprobada por ellos.

Afirmación verificable de IA: «El ejemplo produce 6 y ambos motores respetan el mismo desempate». Se verifica con casos parametrizados, 500 arreglos aleatorios y un oráculo exhaustivo independiente para 1092 arreglos pequeños; el resultado se consulta en el JSON QA real. También se verifica por inspección que fuerza bruta usa suma acumulada en dos bucles. El equipo debe reproducir la verificación y registrar responsable y fecha.

Descartado durante el diseño: inicializar el máximo en cero (falla para todos negativos), recalcular sumas en cada intervalo (introduce costo cúbico), tiempos ficticios, atribuciones simuladas y uso de la web como sustituto del PDF. No se afirma haber medido cobertura.

Pendiente: cada integrante debe entender y defender su parte, rehacer o validar la demostración de forma independiente y redactar la conclusión crítica. El PDF de asignación prohíbe delegar demostración, conclusión o juicio crítico; la propuesta asistida es solo material de trabajo y no constituye evidencia de autoría o comprensión del equipo.

8. Reproducibilidad

Desde la raíz del repositorio: crear entorno virtual, instalar requirements.txt, ejecutar python run_pipeline.py y después python scripts/build_report.py. Consultar README para comandos PowerShell y Linux/macOS. El pipeline se detiene si QA falla y guarda CSV, metadatos, resumen y dos figuras. El script del reporte genera un PDF borrador a partir de esa evidencia.

Semilla experimental y diferencial: 20261105. Deben preservarse los artefactos entregados junto con el commit final. Un commit de metadatos identifica el código ejecutado, no necesariamente el commit posterior que añade los resultados. El hash SHA-256 complementa esa identificación si no había commit. Los tiempos varían entre ejecuciones; reproducibilidad significa mismo método y entradas, no tiempos idénticos.

Distribución de responsabilidades y ramas en docs/team_contributions.md. No hay nombres reales ni revisiones académicas registradas todavía. La bitácora requiere entrega PDF de 4 a 8 páginas, repositorio o comprimido, declaración tecnológica y presentación del equipo. La aprobación del informe y la autenticación para publicación son tareas humanas pendientes.

Repositorio: https://github.com/BryanS200OK/Equipo_5_Bitacora_sub_arreglo

Fuente académica: Asignación de proyectos por equipo y Bitácora 1, Profa. Msc. Sosa, Cohorte X, 2026-III; PDF suministrado por el usuario. La solicitud completa de desarrollo se conserva en docs/user_request.txt.